

Module 02: Variables and Types

You know what a variable is. What you need from Java is narrower: which types exist, which of them are objects and which are not, and where the boundary between those two worlds will trip you up. That boundary is the whole module.

What You'll Learn

- The split between primitives and references, and why Java still has it
- Five numeric behaviours that surprise people, none of which are bugs
- Autoboxing, and the two ways it bites
- Why `==` lies to you about objects, and what to use instead
- Strings as immutable objects, plus text blocks
- `var`, what it does, and where it stops helping

Coming From Another Language

In Python and JavaScript everything is an object, including numbers. Java keeps two separate worlds, and you will feel the seam.

	Python / JS	Java
<code>3</code>	an object	a primitive, 32 bits, no allocation
Declaring	<code>x = 3</code>	<code>int x = 3;</code> or <code>var x = 3;</code>
Changing type later	fine	impossible, the type is fixed at compile time
Integer overflow	grows the number	wraps around silently
<code>"a" == "a"</code>	compares content	compares <i>identity</i> , use <code>.equals</code>
Truthiness	<code>if (items)</code> works	only <code>boolean</code> is allowed in <code>if</code>

That last one deserves a note. Java has no truthiness. `if (someString)` does not compile. You write `if (!someString.isEmpty())`, and the verbosity is the point: there is no rule to memorise about which values count as false.

The Lesson

Two kinds of value

Eight primitive types exist, and the list is closed forever:

boolean	int	long	double
char	byte	short	float

A primitive variable holds the value. Everything else in Java is a **reference**: the variable holds the address of an object living somewhere else.

The difference is invisible until you copy something. See [TwoKindsOfValue.java](#):

```
int[] original = {1, 2, 3};
int[] alias = original; // copies the reference, not the array
alias[0] = 999;
// original[0] is now 999
```

Copying a primitive copies the value. Copying a reference copies the label, and both labels point at the same object. Module 04 returns to this when we look at what happens to method arguments.

In practice you will use `int`, `long`, `double`, `boolean` and `char`. The other three exist mostly for memory-constrained situations you are unlikely to meet.

Numbers will surprise you

Run [NumbersWillSurpriseYou.java](#). Real output:

```
7 / 2 = 3
7 / 2.0 = 3.5
Integer.MAX_VALUE + 1 = -2147483648
Math.addExact says: integer overflow
0.1 + 0.2 = 0.30000000000000004
7 % -2 = 1
-7 % 2 = -1
Math.floorMod(-7, 2) = 1
'A' + 1 = 66
```

Five things are happening there:

1. **Integer division discards the remainder.** `7 / 2` is `3`. Make one operand a `double` and you get `3.5`. When both operands are variables you cannot see this coming, which is what makes it dangerous.
2. **Overflow wraps silently.** An `int` is 32 bits, and adding one to the maximum rolls to the minimum with no exception. `Math.addExact` complains instead, when you would rather know.
3. **`double` cannot hold most decimals exactly.** `0.1` is a repeating fraction in binary. Never store money in a `double`. Use a `long` of cents, or `BigDecimal`.
4. **`%` keeps the sign of the left operand.** `-7 % 2` is `-1`, not `1`. If you wanted the mathematical modulo, that is `Math.floorMod`.
5. **`char` is a number.** `'A' + 1` is `66`, because `+` promoted the char to an int. Cast back with `(char)` to get `B`.

Autoboxing, and its two bites

Every primitive has an object twin: `int` / `Integer`, `double` / `Double`, and so on. Java converts between them automatically. That is autoboxing, and it is genuinely convenient until these two moments.

Bite one, `==` on boxed values. From `TheAutoboxingTrap.java`:

```
127 == 127 : true
128 == 128 : false
```

Same code, same kind of value, opposite answer. Java caches `Integer` objects for `-128..127`, so inside that range `==` compares an object to itself. Outside it, you get two objects and `==` correctly reports that they are different objects.

Nothing is broken. `==` on objects has always meant "the same object", and the cache was hiding that from you.

The rule: `==` for primitives, `.equals()` for objects.

Bite two, unboxing null. `NullUnboxing.java` crashes on purpose:

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "java.lang.Integer.intValue()" because the return value of
  "java.util.Map.get(Object)" is null
```

`Map.get` returns `null` for a missing key, and assigning that to an `int` forces Java to call `intValue()` on nothing. Notice how specific that message is: it names the call that returned null *and* the call that failed on it. Modern Java NPEs do a lot of the debugging for you.

Strings

Strings are objects, and they are **immutable**. No method ever changes the string you called it on. From `StringsAndTextBlocks.java`:

```
String name = "ada";
name.toUpperCase();           // computes "ADA" and throws it away
name = name.toUpperCase();    // you have to keep the result
```

The first line is a real mistake people make, and nothing warns you.

The `==` trap applies here too, for the same reason as `Integer`:

```
literal == literal : true      // the compiler pools identical literals
literal == new     : false     // new String() forces a separate object
equals           : true
```

One habit worth forming immediately: put the literal on the left.

```
"java".equals(maybeNull)    // false
maybeNull.equals("java")   // NullPointerException
```

Text blocks give you multi-line strings without escaping:

```
String json = """
    {
        "name": "Ada"
    }""";
```

Java strips the common leading indentation, measured from the least-indented line, so the block lines up with your code without that whitespace ending up in the value.

var

`var` asks the compiler to work out the type from the right-hand side. Read this part carefully, because it is the thing people get wrong:

`var` is not dynamic typing. The variable still has exactly one type, fixed forever, checked at compile time. You just did not type it out.

```
var x = "hello";
x = 42;    // error: incompatible types: int cannot be converted to String
```

It earns its keep when the type is long and already visible:

```
var scores = new HashMap<String, List<Integer>>();
```

It hurts when the right-hand side does not tell you what you got:

```
var result = service.process(input);    // now go look up process()
```

Three things it cannot do, all compile errors:

```
var a;           // nothing to infer from
var b = null;    // null belongs to every reference type
var c = () -> 1; // a lambda needs a target type
```

`VarCannotInferNull.java` demonstrates the second one. It is also local variables only. Fields, parameters and return types must still be spelled out, which keeps APIs readable.

Run It

```
java modules/02-variables-and-types/examples/TwoKindsOfValue.java
java modules/02-variables-and-types/examples/NumbersWillSurpriseYou.java
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java
java modules/02-variables-and-types/examples/StringsAndTextBlocks.java
java modules/02-variables-and-types/examples/VarAndInference.java

# These two fail on purpose. Read what they say.
java modules/02-variables-and-types/examples/NullUnboxing.java
java modules/02-variables-and-types/examples/VarCannotInferNull.java

./scripts/verify-examples.sh modules/02-variables-and-types
```

Common Mistakes

Using `==` to compare Strings. It works in your test with short literals and fails in production with strings built at runtime. This is the single most common Java bug written by newcomers. Use `.equals`.

Storing money in a `double`. `0.1 + 0.2` is not `0.3`. Use a `long` of cents or `BigDecimal`. Accounting systems have been written the wrong way and the discrepancy shows up months later.

Forgetting that String methods return a new String. `name.strip();` on its own line does nothing at all.

Letting `Map.get` unbox into an `int`. Any missing key becomes a `NullPointerException`. Use `getOrDefault`, or keep it as an `Integer` and check.

Boxing inside a hot loop.

```
Long sum = 0L; // a million allocations
for (long i = 0; i < 1_000_000; i++) sum += i;
```

Changing `Long` to `long` removes every allocation. The two versions look almost identical, which is exactly why this is worth knowing.

Writing `long big = 9000000000;` without the `L`. The literal is parsed as an `int` first, and it does not fit, so it fails to compile.

Key Takeaways

- **Two worlds.** Primitives hold values, references hold addresses. Copying a reference copies the label, not the object.
- `==` asks "same object", `.equals` asks "same content". On anything that is not a primitive, you want `.equals`.

- **The Integer cache makes `==` look correct below 128.** That is what makes it dangerous, not the boundary itself.
- **Numbers are fixed-width.** Integer division truncates, overflow wraps, and `double` cannot hold most decimals exactly.
- **Strings never change.** Every method returns a new one; keep the result.
- **`var` is inference, not dynamic typing.** The type is still fixed and still checked. Use it when the type is obvious on the same line.
- **Java has no truthiness.** Only a `boolean` goes in an `if`, and that is a feature.

Homework

[homework/README.md](#)

Build a small receipt calculator that gets the money handling right, then reproduce two of this module's traps on purpose. Reference solution is in [solutions/02-variables-and-types/](#).